
Royco Dusk

ROYCO PROTOCOL · TECHNICAL SPECIFICATION · v2

Dusk extends Royco Dawn so the Junior Tranche transforms both risk and liquidity profiles. Coverage capital is deployed as a Gyroscope E-CLP liquidity position on Balancer V3, paired with Senior Tranche shares, providing instant secondary liquidity with asymmetric concentrated depth that reprices on oracle rate.

Core Problem	Recursive coverage. The Junior Tranche's NAV contains Senior Tranche shares that are themselves backed by the Junior Tranche's NAV.
Resolution	Treat Senior Tranche shares held by the Junior Tranche as transiently exited; price them at pool fraction only, exclude from active claims.
Substrate	Gyroscope E-CLP on Balancer V3 with hook-driven accounting sync and rate provider for oracle-rate repricing. E-CLP parameters (α , β , λ , φ) are immutable per pool.
Asset Layout	Different assets per tranche. The Senior Tranche asset is the underlying; the Junior Tranche asset is the Balancer Pool Token (BPT).

01 What Dusk Does

Royco Dawn splits any priceable asset into two profiles — a Senior Tranche (capital-protected) and a Junior Tranche (first-loss). That solved risk transformation. But credit-worthiness depends on two dimensions: risk profile and liquidity profile. Dawn alone does not address the second.

Dusk extends the Junior Tranche to do both. Instead of holding plain stablecoins as coverage, the Junior Tranche deploys its capital into a Gyroscope E-CLP pool on Balancer V3, paired against Senior Tranche shares:

- **Risk transformation.** Coverage still flows from the Junior Tranche to the Senior Tranche exactly as in Dawn.
- **Liquidity transformation.** The Junior Tranche's pool position gives Senior Tranche holders an instant secondary market with asymmetric concentrated depth around oracle price.

All quantities in this document are in NAV units (the kernel's native value denomination, e.g. USD) unless otherwise noted.

02 The Recursive Coverage Problem

Putting Senior Tranche shares into the Junior Tranche's pool creates a circular dependency. All Senior Tranche shares are protected pro-rata by the Junior Tranche's NAV. But if that NAV is partly composed of Senior Tranche shares, those shares are themselves protected by the NAV that contains them.

Dusk eliminates this by treating Senior Tranche shares inside the Junior Tranche's pool as transiently exited. They are part of the Junior Tranche's asset side, not active senior claims, and should not be covered.

03 AMM Substrate

Three properties drive the substrate choice:

- **Asymmetric concentrated liquidity.** E-CLP concentrates all pool capital within a configurable price range $[\alpha, \beta]$, with λ and φ controlling depth distribution. Unlike symmetric pools, E-CLP skews depth toward the sell-shares-for-stables direction — exactly where exits happen.
- **Oracle-rate repricing.** E-CLPs natively support Balancer V3 rate providers. As the share's NAV changes, the rate provider shifts the pool's entire pricing curve automatically. No arbitrage is needed to rebalance. The $[\alpha, \beta]$ bounds move with the rate.
- **Hook-driven lifecycle callbacks.** V3's hook system provides clean `onBefore` / `onAfter` callbacks on every state-mutating pool operation, giving the protocol clean accounting boundaries without mid-transaction state inconsistencies.

Pool Configuration

Senior Tranche share	→	<code>TokenType.WITH_RATE</code> (rate-oracled)
Quote asset	→	<code>TokenType.STANDARD</code> or <code>WITH_RATE</code>
Rate provider	→	Royco Dusk Kernel (both tokens)
Pool type	→	Gyroscope E-CLP (<code>GyroECLPPool</code>)
Hook contract	→	<code>RoycoDuskBalancerV3Hook</code>
Parameters	→	$\alpha, \beta, \lambda, \varphi$ (immutable per pool)

E-CLP parameters are immutable Solidity constants set at deployment. α and β define the price range; λ controls concentration (higher = more depth near par); φ controls rotation (skews liquidity toward the exit direction). Parameter selection is per-market, tuned to the underlying asset's expected volatility and exit demand profile.

The quote asset can be a standard stablecoin (USDC) or a yield-bearing stablecoin (sUSDs, srRoyUSDC). When yield-bearing, the quote asset is registered as `TokenType.WITH_RATE` and the kernel serves as its rate provider. The kernel already tracks the quote asset's rate internally to price quote-asset reserves in JT's BPT holdings for NAV accounting, so exposing it through V3's rate provider interface adds no new state or complexity.

Deployment uses the existing Gyroscope E-CLP factory on Balancer V3. The hook contract whitelists this factory in its `onRegister` callback. No custom factory or governance is required. BPT is gauge-eligible via standard Balancer governance vote; Gyroscope E-CLP pools have established gauge precedent.

04 Kernel Asset Layout

Dusk is the first non-identical-asset Dawn kernel: the Senior Tranche asset $\neq$ the Junior Tranche asset.

```
SENIOR_TRANCHE_ASSET      = underlying yield-bearing asset (e.g. sUSDe)
stOwnedYieldBearingAssets  raw underlying held by the kernel

JUNIOR_TRANCHE_ASSET      = Balancer Pool Token (BPT)
jtOwnedYieldBearingAssets  BPT held by the kernel
```

All Senior Tranche asset lives in the kernel, not in the pool. The pool holds quote asset and Senior Tranche shares; the shares are claims that resolve through the Senior Tranche's redemption path.

The pool is open to external LPs. The kernel's accounting uses pro-rata BPT ownership ($\text{jtOwnedBPT} / \text{totalBPTSupply}$) to compute JT's share of pool reserves. External LP activity does not affect tranche accounting.

05 The Mechanism

Effective Supply

The kernel tracks one new state variable: `internalShares` — the count of Senior Tranche shares the Junior Tranche's pool holds (pro-rata to JT's BPT ownership), as of the last accounting sync. These shares are treated as transiently exited: they do not accrue coverage and do not share in the Senior Tranche's effective NAV.

The denominator for deposits and redemptions becomes:

```
effectiveSupply = totalSupply - internalShares
```

External holders — the only ones with active claims — redeem at the Senior Tranche's effective NAV divided by effective supply. Internal shares cannot claim on the Senior Tranche's effective NAV (that would be recursive). The Junior Tranche realises value from those shares only when BPT is unwound.

Quoter

The quoter computes each tranche's raw NAV against the cached partition, ensuring it is non-recursive and idempotent within a sync. `internalShares` and `quoteReserves` are already pro-rata to JT's BPT ownership (computed from the pool's reserves scaled by `jtOwnedBPT / totalBPTSupply`). The formulas use these pro-rata values directly:

```
ST_RAW_NAV = assetPrice × stAssets × ( stShares - internalShares ) / stShares  
  
JT_RAW_NAV = assetPrice × stAssets × internalShares / stShares  
            + quotePrice × quoteReserves
```

Here `assetPrice` is the per-unit NAV price of the underlying, `stAssets` is the kernel-held underlying, `stShares` is the total Senior Tranche supply. `internalShares` is JT's pro-rata share of the pool's Senior Tranche share reserves. `quoteReserves` is JT's pro-rata share of the pool's quote-asset balance. `quotePrice` is the per-unit value of the quote asset (1 for standard stablecoins, the kernel's rate for yield-bearing). No outer pro-rata factor is needed; the BPT ownership fraction is already embedded in `internalShares` and `quoteReserves`. **Conservation holds by construction:**

```
ST_RAW_NAV + JT_RAW_NAV = assetPrice × stAssets + quotePrice × quoteReserves (at sole LP)
```

06 Sync Flow

Between any two accounting syncs, two independent things happen: underlying PnL (the yield-bearing asset accrued yield or depreciated) and BPT composition shift (swaps, adds, or removes changed the ratio of shares to quote asset inside the pool). V3's hook system separates these cleanly via two callbacks per operation.

onBefore: Underlying PnL + Rate Refresh

Fires before every swap, add, and remove. The hook calls the kernel to process any underlying asset PnL that has accumulated since the previous sync. The kernel runs the PnL waterfall at the existing partition (the `internalShares` set by the previous operation's `onAfter`). After the hook returns, V3's Vault reloads balances and rates via `poolData.reloadBalancesAndRates()`, so the pool's bonding-curve math uses the freshly updated rate from that point forward.

onAfter: Recomposition + Composition Shift

Fires after every swap, add, and remove. The operation has now executed and the pool's composition has changed. The hook calls the kernel, which reads the new pool state, runs the recomposition (partition update), and accounts for the composition shift this operation caused.

Why Two Hooks Per Operation

Processing underlying PnL and composition shifts in the same accounting pass can produce different results depending on how many events accumulate between syncs. By splitting them into two callbacks, each sync sees exactly one type of state change. The `onBefore` always captures PnL at the existing partition; the `onAfter` always captures the composition shift and updates the partition. This eliminates order-dependent accounting artifacts.

Senior deposits, senior redemptions, and JT redemptions that go through the kernel directly (not through the pool) use the same accounting logic. The hook is not involved for these operations.

Recomposition

Within the `onAfter` callback, the kernel reads the post-operation pool composition to obtain `newInternalShares` (JT's pro-rata share of the pool's Senior Tranche share reserves). Define the scaling factor as the ratio of new external share count to old:

```
scalingFactor = ( stShares - newInternalShares ) / ( stShares - internalShares )
```

Senior Tranche checkpoints (scaled)

```
ST_RAW_NAV      = LAST_ST_RAW_NAV      × scalingFactor
ST_EFFECTIVE_NAV = LAST_ST_EFFECTIVE_NAV × scalingFactor
ST_IL           = LAST_ST_IL           × scalingFactor
```

Junior Tranche checkpoints

```
JT_RAW_NAV      = LAST_JT_RAW_NAV
JT_EFFECTIVE_NAV = ST_RAW_NAV + JT_RAW_NAV - ST_EFFECTIVE_NAV
JT_IL           = LAST_JT_IL           × scalingFactor
```

After writing the recomposed checkpoints, set `internalShares = newInternalShares`.

PnL Waterfall

Run Dawn's existing accounting waterfall with the quoter's current-price raw NAVs against the recomposed checkpoints. The waterfall processes deltas in this order:

1. Junior Tranche delta
Loss: JT absorbs; excess spills to Senior Tranche IL
Gain: Recover Senior Tranche IL first; JT accrues residual
2. Senior Tranche delta
Loss: JT covers via $\min(\text{loss}, \text{JT_EFFECTIVE_NAV})$
Gain: Recover ST IL, then JT IL, then distribute yield via YDM

This ordering ensures the Junior Tranche's delta (including any coverage-premium cost from composition shifts) is fully processed before the coverage step runs.

07 Derivation of the Recomposition

Core Principle

The recomposition has two distinct jobs. First, remove phantoms from the deltas the waterfall will see — changes in raw NAV caused by share reclassification with no corresponding economic event. Second, preserve the Dawn invariant that links impermanent-loss accumulators to the cross-tranche claim. Both jobs are necessary; together they fully account for the partition shift.

Why the Senior Tranche Has a Phantom

Without recomposition, at unchanged prices with shares entering the pool:

$$\text{deltaST} = - (\text{assetPrice} \times \text{stAssets} \times \text{deltaInternal} / \text{stShares}) < 0$$

The waterfall would interpret this as a Senior Tranche loss and trigger Junior Tranche coverage. But nothing economic happened — shares were relabeled with no offsetting cash flow on the Senior Tranche side. This is a pure phantom, removed by scaling.

Why the Junior Tranche Has No Phantom in Raw NAV

The same reclassification adds share backing to the Junior Tranche's pool. But unlike the Senior Tranche side, every reclassification on the Junior Tranche side is paired with a real swap: quote asset left the pool to pay for the incoming shares. At unchanged prices:

$$\text{deltaJT} = \text{deltaInternal} \times (\text{perShareRaw} - \text{swapPrice})$$

Where $\text{perShareRaw} = \text{assetPrice} \times \text{stAssets} / \text{stShares}$ and swapPrice is the average execution price. The reclassification component and the cash flow component always occur together:

- **At par** ($\text{swapPrice} = \text{perShareRaw}$): $\text{deltaJT} = 0$. Perfect cancellation.

- **Off par:** ΔJT equals the execution residual — a real economic gain or loss.

There is no phantom in ΔJT . Every reclassification on the Junior Tranche side is accompanied by a real cash flow. Therefore the Junior Tranche's raw NAV checkpoint is left unchanged. ■

Why Both Impermanent-Loss Accumulators Scale

The phantom-removal argument applies to raw NAV because JT_RAW is a quoter output. The impermanent-loss accumulators are state variables that track outstanding recovery claims, and the right rule for them depends on the mechanics of how external supply changes.

What JT_IL represents. JT_IL is JT's claim that future Senior Tranche asset appreciation should priority-flow back to JT, in compensation for past coverage. The recovery rate per dollar of $stAsset$ appreciation is proportional to external-share count.

Why Dusk scales JT_IL on recomposition. When the partition shifts via the pool, the exiting shares move internal, not gone. $stAssets$ is unchanged. The $stAsset$ appreciation that previously flowed entirely through ΔST now splits: a portion flows through ΔST (proportional to remaining external), and the rest flows directly into JT_RAW as appreciation on the newly internal shares JT owns through the pool. If JT_IL stayed absolute, JT would recover the full original claim from the external channel and also capture the internal-channel appreciation: a double-count. Scaling by the external-share ratio prevents it.

What this preserves. Scaling JT_IL preserves the invariant that JT recovers exactly its original coverage debt, no more, no less. ST_IL scales for the analogous reason.

Consequence for Junior Tranche Effective NAV

After scaling the Senior Tranche and leaving the Junior Tranche's raw NAV unchanged, conservation gives:

$$\begin{aligned} JT_EFFECTIVE_NAV &= LAST_JT_EFFECTIVE_NAV \\ &+ (LAST_ST_EFFECTIVE_NAV - LAST_ST_RAW_NAV) \\ &\times \Delta Internal / externalShares \end{aligned}$$

Where $externalShares = stShares - internalShares$ (the old external count). When coverage has been applied, the Junior Tranche's effective NAV increases. This is not phantom — it reflects fewer external shares to protect.

08 Delta Composition

The Senior Tranche delta captures purely the change in value of the underlying asset between syncs, scaled to the external fraction. Pool mechanics never touch the kernel's underlying balance, so they cannot bleed into this delta.

The Junior Tranche delta captures the sum of three real economic events:

- **Yield or depreciation on internal shares.** Proportional to the count of internally-held Senior Tranche shares.
- **Pool-making revenue.** Swap fees, slippage, and (when coverage is active) the oracle-rate premium cost from shares entering above raw backing.
- **Bonding-curve execution residuals.** The difference between actual E-CLP execution and the oracle rate.

All three flow through Dawn’s existing waterfall unchanged.

Swaps Cannot Produce Senior Tranche Loss

Claim. At constant underlying prices, no swap — regardless of direction or size — can produce $\text{deltaST} < 0$. Swaps cannot trigger coverage from their own mechanics.

Proof. A swap changes the pool’s internal composition (shares enter or leave the pool), shifting `internalShares`. The recomposition scales the Senior Tranche checkpoint by `scalingFactor = newExternal / oldExternal`. After recomp:

```
ST_RAW_checkpoint = assetPrice × stAssets × oldExternal / stShares
                  × newExternal / oldExternal
                  = assetPrice × stAssets × newExternal / stShares
```

The quoter’s current-price computation is:

```
ST_RAW_current    = assetPrice × stAssets × newExternal / stShares
```

These are identical. Therefore $\text{deltaST} = \text{ST_RAW_current} - \text{ST_RAW_checkpoint} = 0$ exactly.

The cancellation holds because swaps change only the pool’s internal composition. They do not change `stAssets` (the kernel’s underlying balance is untouched by pool operations), `stShares` (no shares are minted or burned), or `assetPrice` (assumed constant within a sync). The recomp scaling factor tracks the partition shift exactly, leaving zero residual on the Senior Tranche side.

Corollary: the only source of $\text{deltaST} \neq 0$ is a change in `assetPrice` between syncs — a real economic event, not a pool mechanic. Swaps produce `deltaJT` only (fee + σ). ■

09 Coverage Sizing: Dynamic

In Dawn, β (BETA) is a static per-market constant representing the Junior Tranche’s sensitivity to Senior Tranche downside, scaled to WAD (10^{18} fixed-point). $\beta = 0$ means uncorrelated; $\beta = \text{WAD}$ means fully correlated.

In Dusk, the Junior Tranche's portfolio is a shifting mix of quote asset (uncorrelated) and internal Senior Tranche shares (fully correlated). Dusk computes β per-sync as a composition-weighted average:

$$\text{BETA} = \frac{(\text{assetPrice} \times \text{stAssets} \times \text{internalShares} / \text{stShares} \times \text{WAD})}{\text{JT_RAW_NAV}}$$

BETA is bounded above by WAD, attained when the pool holds zero quote asset. For single-asset kernels (all existing Dawn markets), this reduces to the configured static β with no behaviour change. The kernel updates the accountant's beta parameter before each sync.

10 Coverage Invariant Under Pool Swaps

This section proves that, at constant Senior Tranche asset prices, swap activity through the pool cannot worsen utilization — unconditionally for the trade direction relevant to exit liquidity, and conditionally for the opposite direction with explicit bounds.

The argument has three steps. First, the coverage numerator depends only on stAsset valuation, not on how shares are partitioned (Lemma 1). Second, swap geometry produces a slippage residual whose sign is fixed by trade direction (Lemma 2). Third, every swap also collects a fee that reliably exceeds adverse residuals across well-parameterized operating ranges (Theorem).

Definitions

A rate-oracled E-CLP pool has a rate-adjusted balance point where the quote reserves equal the rate-adjusted value of the Senior Tranche share reserves. Relative to this balance point:

- A balancing trade moves the pool toward the balance point. The bonding curve pays a small premium above the oracle rate to incentivise rebalancing.
- An imbalancing trade moves the pool away from the balance point. The bonding curve pays below the oracle rate, penalising further imbalance.

The exit-liquidity case Dusk is built for is consistently imbalancing: Senior Tranche holders selling shares into a pool whose share side is already accumulating from prior exits.

Lemma 1 — Partition-Invariant Coverage Numerator

Claim. The coverage numerator $\text{ST_RAW_NAV} + \beta \cdot \text{JT_RAW_NAV}$ collapses to a quantity Π that depends only on stAsset valuation, not on how shares are split between external and internal supply.

Intuition. Coverage exists to absorb stAsset depreciation. The pool's quote reserves are insulated from stAsset price moves. Whether a Senior Tranche share is held externally or sits inside the pool, it represents the same claim on stAssets. Π is therefore the total Senior

Tranche-asset exposure, invariant to the partition.

Proof. Substituting the dynamic- β definition into the coverage numerator:

$$\begin{aligned} & \text{ST_RAW_NAV} + \text{JT_RAW_NAV} \times \beta / \text{WAD} \\ &= \text{perShareRaw} \times \text{externalShares} + \text{perShareRaw} \times \text{internalShares} \\ &= \text{perShareRaw} \times \text{stShares} \\ &= \text{assetPrice} \times \text{stAssets} \\ &:= \Pi \end{aligned}$$

The quote-asset term in JT_RAW_NAV cancels because β weights JT_RAW_NAV by the underlying-backed fraction. Π is constant between syncs, and $\text{util} = \text{COVERAGE} \times \Pi / \text{JT_EFFECTIVE_NAV}$ depends only on JT_EFFECTIVE_NAV . ■

Lemma 2 — Sign and Magnitude of the Slippage Residual

Setup. A swap of NAV size t executes against the bonding curve. Define $\sigma(t, \text{state})$ as the difference between the pool NAV after the swap and what it would have been at the oracle rate (no slippage).

Sign claim. $\sigma \geq 0$ for imbalancing trades and $\sigma \leq 0$ for balancing trades.

Intuition for sign. The E-CLP bonding curve (like all concentrated-liquidity invariants) is more curved away from the balance point and flatter near it. An imbalancing trade pushes into the curved region: the trader pays a premium, accruing additional NAV to the pool. A balancing trade does the reverse. The geometry is unconditional — it follows from concavity of the invariant alone.

Magnitude claim (balancing case). For trades that pull the pool toward balance:

$$|\sigma(t, \text{state})| \leq \kappa(\text{state}) \times t^2$$

Where κ (kappa) is the local curvature of the invariant at the current operating point. For any smooth concave bonding curve, the slippage residual is quadratic in trade size t — marginal slippage grows linearly, so total slippage integrates to t^2 . The curvature κ captures all pool-specific geometry:

- **E-CLP:** κ is a function of λ (concentration), the current position on the ellipse, and the invariant value. Higher λ compresses the curve's active region, reducing κ near the center and increasing depth. At the balance point, κ approaches zero; near the $[\alpha, \beta]$ boundaries, κ grows as liquidity thins.
- **Stableswap (for comparison):** $\kappa = K / (A \cdot D)$, where A is amplification and D is pool size. The E-CLP generalizes this.

κ is tunable through E-CLP parameter selection ($\alpha, \beta, \lambda, \varphi$). For the operating ranges Dusk targets — arbitrage keeps the pool within a few percent of balance — κ stays small.

Theorem — Utilization Non-Increasing Under Swaps

Claim. At constant Senior Tranche asset prices, every swap satisfies one of:

- **Hard guarantee:** $\sigma \geq 0$, fee adds further to $JT_EFFECTIVE_NAV$, utilization strictly decreases.
- **Soft guarantee:** $\sigma < 0$ but the fee dominates, so $JT_EFFECTIVE_NAV$ still grows and utilization decreases.
- **Bounded breach:** $\sigma < 0$ and the residual exceeds the fee. Utilization can rise, but only by an amount bounded by Lemma 2.

Proof. The change in $JT_EFFECTIVE_NAV$ from a single swap decomposes into two terms:

$$\Delta JT_EFF = \text{swapFeeRate} \times t + \sigma(t, \text{state})$$

where swapFeeRate is the pool's static swap fee (set at deployment, configurable by the swap fee manager). By Lemma 1, Π is unchanged. Since util is monotonically inverse in $JT_EFFECTIVE_NAV$:

$$\text{util_new} \leq \text{util_old} \Leftrightarrow \Delta JT_EFF \geq 0 \Leftrightarrow \text{swapFeeRate} \times t \geq |\sigma(t, \text{state})|$$

Imbalancing trades. By Lemma 2, $\sigma \geq 0$. Both terms are non-negative; utilization strictly decreases. Hard guarantee.

Balancing trades. By Lemma 2, $\sigma \leq 0$ with magnitude bounded by $\kappa \times t^2$. The fee dominates whenever:

$$\text{swapFeeRate} \times t \geq \kappa \times t^2$$

Solving for t , the threshold below which utilization is guaranteed non-increasing:

$$t \leq \text{swapFeeRate} / \kappa(\text{state})$$

For trades exceeding the threshold, the worst-case utilization increase is:

$$\Delta \text{util} \leq (\kappa \times t^2 - \text{swapFeeRate} \times t) / JT_EFF$$

Bounded, predictable, and tunable through E-CLP parameter selection. ■

Imbalancing trades — the dominant flow during exit pressure — are unconditionally safe regardless of parameters. In Balancer V3, the full swap fee accrues to BPT holders (no admin-fee split at the pool level), so the JT captures 100% of swap fee revenue through its BPT ownership. This raises the soft-guarantee threshold relative to substrates that route a portion of the fee to protocol revenue.

11 Junior Tranche Deposits

Junior Tranche deposits are dual-asset. The depositor contributes both underlying and quote asset. The deposit is atomic, performed in a single transaction via the Junior Tranche vault.

The Sequence

Every Junior Tranche deposit of X underlying and Y quote asset follows these steps, atomic in a single transaction:

- **Pull quote asset and underlying from the depositor.**
- **Deposit X underlying into the Senior Tranche vault.** The kernel mints Senior Tranche shares. The coverage check passes because JT capacity was already sufficient or is being grown in this same transaction.
- **Call `addLiquidityUnbalanced([shares, Y], ...)`** V3 accepts the shares and quote asset, returning BPT. If the amounts match the pool's current ratio, V3 routes through the proportional path automatically (zero swap fee).
- **Mint JT shares to the depositor** proportional to BPT received and current JT effective NAV.

The executor ends with $X + Y$ NAV worth of Junior Tranche shares. The hook fires `onBefore` and `onAfter` around the `addLiquidityUnbalanced` call, ensuring accounting is synced.

Bootstrap (First Deposit)

Normal senior share minting requires junior capital to already exist (coverage check). But junior capital in Dusk is BPT, which requires senior shares to be in the pool. This is circular.

The bootstrap resolves this with a permissioned kernel call that bypasses the coverage check for the initial mint. All minted shares go directly into the pool as internal supply. With zero external senior holders, there is no coverage obligation to enforce.

Bootstrap flow (single atomic transaction via the JT vault):

- Pull quote asset and underlying from the bootstrapper.
- Mint senior shares from the underlying, bypassing the coverage check (permissioned bootstrap path only).
- Call `Router.initialize(pool, [shares, quoteAmount], ...)`. V3's first-deposit semantics compute the invariant directly from the initial balances. The rate provider returns `perShareRaw` during this call (the all-internal fallback, since `effectiveSupply = 0`).
- BPT minted to the JT vault. Kernel writes initial checkpoints. Pool state: all shares internal, zero external.

The first external senior interaction transitions the system into normal operation, where every subsequent senior mint is subject to the standard coverage check.

Why Every Deposit Follows This Pattern

- **Pool composition tracks exit pressure exclusively.** The only flow that shifts pool composition toward share-heavy is Senior Tranche holders selling shares into the pool — exactly Dusk’s exit-liquidity use case. Pool imbalance becomes a clean signal of unmet exit demand.
- **Rebalancing is incentivized for free.** The depositor’s chosen (X, Y) ratio determines whether the deposit is balancing or imbalancing. V3’s existing fee structure charges more for imbalancing deposits. Depositors maximizing JT shares per dollar prefer to bring the scarcer asset, naturally pushing the pool back toward balance.
- **Coverage scaling is automatic.** When the depositor brings X underlying, the kernel mints exactly X new shares and absorbs them into the pool as internal supply. JT_EFFECTIVE_NAV grows by X + Y. Capital scales coherently across all three sides.
- **External Senior Tranche capacity is preserved.** The deposit mints shares and absorbs them back into pool inventory as internal supply. Net change to external share count is zero. JT_EFFECTIVE_NAV grows by X + Y. The cap on new external Senior Tranche deposits strictly expands.

12 Junior Tranche Redemption

Redemption delivers underlying assets (quote asset and Senior Tranche asset) without disturbing external Senior Tranche holders’ per-share effective value. Dawn’s claim decomposition splits the total NAV into four components:

```
stClaimOnJTRawNAV    = max( 0, ST_EFFECTIVE_NAV - ST_RAW_NAV )
jtClaimOnSTRawNAV    = max( 0, JT_EFFECTIVE_NAV - JT_RAW_NAV )
stClaimOnSelfRawNAV  = ST_RAW_NAV - jtClaimOnSTRawNAV
jtClaimOnSelfRawNAV  = JT_RAW_NAV - stClaimOnJTRawNAV
```

For a Junior Tranche redeemer claiming a fraction f of the tranche ($\text{redeemerShares} / \text{jtSupply}$), the total NAV delivered is:

```
NAV_to_redeemer = f × ( jtClaimOnSelfRawNAV + jtClaimOnSTRawNAV ) = f × JT_EFFECTIVE_NAV
```

These components are settled differently:

- **jtClaimOnSelfRawNAV portion.** Delivered as Junior Tranche asset (BPT, unwrapped via `removeLiquidityProportional` to quote asset + Senior Tranche shares).
- **jtClaimOnSTRawNAV portion.** Delivered as Senior Tranche asset directly from the kernel’s underlying balance (Dawn’s existing pattern, unchanged in Dusk).

BPT Unwrap Mechanism

The kernel unwraps a NAV-weighted fraction of its BPT holdings, not a flat pro-rata of redeemer shares:

$$\text{bptToWithdraw} = f \times (\text{jtClaimOnSelfRawNAV} / \text{JT_RAW_NAV}) \times \text{jtOwnedBPT}$$

In the healthy state, $\text{jtClaimOnSelfRawNAV} = \text{JT_RAW_NAV}$, so bptToWithdraw collapses to $f \times \text{jtOwnedBPT}$ — flat pro-rata. In the coverage state, $\text{jtClaimOnSelfRawNAV} = \text{JT_EFF} < \text{JT_RAW_NAV}$, so bptToWithdraw is reduced. The unredeemed fraction stays in the pool to honour future Senior Tranche redemption claims.

Three-step execution

- **Pro-rata pool withdrawal.** Call `Router.removeLiquidityProportional(pool, bptToWithdraw, ...)`. The pool returns proportional quote asset and Senior Tranche shares. Pool composition is preserved exactly.
- **Par-coupled burn.** Burn the Senior Tranche shares received and reduce the kernel's underlying balance by $\text{burned} \times \text{stAssets} / \text{stShares}$. stShares decreases by the burned amount.
- **Transfer.** Deliver quote asset from step 1, Senior Tranche asset from step 2, plus any jtClaimOnSTRawNAV portion drawn directly from the kernel's underlying balance.

Preservation Proofs

External share count

The par-coupled burn removes the same count from both supplies:

$$\begin{aligned} \text{externalShares_new} &= (\text{stShares} - \text{burned}) - (\text{internalShares} - \text{burned}) \\ &= \text{externalShares} \quad \blacksquare \end{aligned}$$

Per-share Senior Tranche effective NAV

ST_EFF is unchanged by JT redemption (the kernel only debits JT_EFF by the redemption amount). External share count is preserved. Therefore the per-external-share effective NAV is preserved exactly:

$$\text{ST_EFF_new} / \text{externalShares_new} = \text{ST_EFF} / \text{externalShares} \quad \blacksquare$$

NAV conservation

The redeemer's BPT fraction times the pool's value yields the $\text{jtClaimOnSelfRawNAV}$ portion; the jtClaimOnSTRawNAV portion is added directly:

$$\text{NAV_to_redeemer} = f \times \text{JT_EFFECTIVE_NAV} \quad \blacksquare$$

13 Senior Tranche Redemption Override

Dawn's default kernel routes `stClaimOnJTRawNAV` (the Senior Tranche's claim on Junior Tranche raw NAV during coverage) through Junior Tranche asset units. In Dusk this is inappropriate: a redeeming Senior Tranche holder should receive Senior Tranche asset, not BPT.

The Dusk kernel overrides the Senior Tranche redemption path:

- `stClaimOnSelfRawNAV` **portion**. Senior Tranche asset from the kernel's underlying balance, with par-coupled accounting.
- `stClaimOnJTRawNAV` **portion (coverage settlement)**. Also delivered as Senior Tranche asset. Sourced by partial BPT unwrap: the kernel calls `removeLiquidityProportional` on enough of JT's BPT to cover this NAV, burns any Senior Tranche shares received (par-coupled), and pays the redeemer in Senior Tranche asset.

This preserves Dawn's coverage semantics (Senior Tranche holders are made whole using Junior Tranche assets) while respecting Dusk's asset structure (Senior Tranche holders never receive BPT).

14 Rate Provider

The Royco Dusk Kernel implements V3's `IRateProvider` interface for both pool tokens. V3's Vault consults the rate provider on every state-mutating operation via `poolData.reloadBalancesAndRates(...)`. For the senior tranche share token, the rate anchors the pool's bonding-curve math to the protocol's per-share NAV. For yield-bearing quote assets, the kernel returns the quote asset's current rate, which it already tracks internally for NAV accounting.

The rate provider has three branches, each handling a different lifecycle state:

```
if stShares == 0 or stAssets == 0: return WAD
```

No senior tranche shares or underlying assets exist yet. Pre-bootstrap. Return 1:1 as a safe default.

```
if effectiveSupply == 0: return perShareRaw
```

Senior shares exist but are all inside the pool (none held externally). This is the state immediately after bootstrap. Rate falls back to the raw per-share value of the underlying (`stAssets / stShares`), without coverage adjustment.

```
return ST_EFFECTIVE_NAV / effectiveSupply
```

Normal operation. The rate is the effective NAV per external share. “Effective” means it accounts for coverage: if the underlying has depreciated, JT absorbs the loss first, so the effective rate stays higher than the raw underlying value.

Because the hook triggers a protocol sync before the Vault reloads rates, the rate provider always returns a freshly computed value. When the quote asset is yield-bearing, the kernel serves as rate provider for both tokens, exposing the quote rate it already tracks for internal NAV accounting.

15 RoycoDuskBalancerV3Hook

The hook contract is registered at pool registration time. The hook’s onRegister callback whitelists the Gyroscope E-CLP factory, enabling deployment through the standard factory path with no custom governance.

The hook serves two roles:

- **Accounting sync.** Calls the kernel’s accounting sync in every onBefore and onAfter callback. The onBefore handles underlying asset PnL and rate refresh at the existing partition. The onAfter handles recomposition (partition update) and composition shift accounting.
- **Bootstrap gate.** Gates initialize to the bootstrapper address, preventing unauthorized first deposits.

The hook contract is immutable post-registration. Runtime configuration lives in the kernel and is read by the hook on each call. All six onBefore/onAfter callback flags are set to true (swap, add, remove). The pool is open to external LPs with no gating on subsequent adds or removes.

16 Open Items

E-CLP parameter selection. α , β , λ , and φ jointly determine the pool’s depth profile, concentration, and directional skew. Each market should choose parameters such that the concentrated range covers the expected trading range of the Senior Tranche share token, with sufficient depth at the exit boundary to absorb realistic exit demand without excessive slippage.

MEV and sandwich resistance. Standard AMM sandwich risk applies. The pool provides liquidity availability, not guaranteed execution at the oracle rate for arbitrary trade sizes.

Senior Tranche redemption gas cost. The override requires partial BPT unwrap when coverage has been applied. Gas profile and slippage tolerance configuration should be reviewed.

Self-liquidation bonus interaction. Dawn’s self-liquidation bonus on Senior Tranche redemption uses Junior Tranche NAV. In Dusk, this requires unwrapping BPT to source the

bonus in Senior Tranche asset; the override path must handle this case explicitly.

Gauge eligibility. BPT from Dusk E-CLP pools should be eligible for Balancer gauges and Aura emissions. Gyroscope E-CLP pools have established gauge precedent; the governance process for Dusk pools should be confirmed.

17 Summary

Dusk turns the Junior Tranche from a pure coverage vehicle into a coverage-and-liquidity vehicle without changing Dawn's core promises. The single new state variable, the recomposition, the effective-supply share math, and the composition-weighted dynamic β break the recursive coverage problem and deliver continuous secondary liquidity for any Senior Tranche on any priceable asset.

The recomposition has two distinct jobs grounded in different invariants. Raw NAV checkpoints follow a structural asymmetry: Senior Tranche reclassification creates a phantom that must be scaled away, while Junior Tranche reclassification is paired with a real swap and leaves no phantom in raw NAV. Impermanent-loss accumulators scale by the external-share ratio because the partition shift creates a dual recovery channel. Scaling preserves the Dawn invariant that JT recovers exactly its original coverage debt without double-counting across both channels.

Junior Tranche deposits are atomic. Each deposit contributes underlying and quote asset in a single transaction, minting Senior Tranche shares and adding liquidity to the pool via `addLiquidityUnbalanced`. Pool composition shifts only when Senior Tranche holders exit through the secondary path, making pool imbalance a clean signal of exit demand. V3's existing fee structure incentivizes rebalancing deposits without protocol intervention. Bootstrap is the empty-pool special case, resolved by a permissioned coverage-check bypass for the initial mint.

Junior Tranche redemption uses a NAV-weighted BPT unwrap via `removeLiquidityProportional` that collapses to flat pro-rata in the healthy state and reduces unwrap volume by JT_EFF / JT_RAW in the coverage state, preserving external Senior Tranche holders' per-share effective NAV. Senior Tranche redemption is routed through a Dusk-specific override that always settles in Senior Tranche asset, never in BPT.

Deployed on Gyroscope E-CLP on Balancer V3, Dusk captures the full swap fee through JT's BPT ownership with no admin-fee split at the pool level. V3's hook system drives real-time accounting sync via two-point callbacks (`onBefore` for underlying PnL at the existing partition, `onAfter` for recomposition and composition shifts), eliminating order-dependent accounting artifacts. The rate provider anchors pricing to the protocol's NAV oracle, refreshed on every operation. The coverage invariant is approximately preserved under swaps: imbalancing trades guarantee non-increasing utilization unconditionally, and balancing trades stay within a bounded residual tunable through E-CLP parameter selection.

Royco Dusk Technical Specification — v2. Subject to refinement during audit.
Properties verified against the royco-dawn codebase. Not financial advice.